

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

О Т Ч Е Т

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6.2
«Работа с БД в СУБД MongoDB»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Смирнов Фёдор Евгеньевич
Факультет прикладной информатики
Группа К3240
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2024
Преподаватель Говорова Марина Михайловна

Санкт-Петербург,
2026

Цель работы

Цель работы — овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Практическое задание

В ходе выполнения лабораторной работы требовалось выполнить практические задания по работе с базой данных документов в СУБД MongoDB. В работе использовалась база данных learn и коллекции unicorns, towns, habitats и numbers.

В рамках лабораторной работы были выполнены:

1. вставка документов в коллекцию;
2. выборка документов с использованием find, findOne, sort, limit, skip и проекций;
3. работа с массивами и логическими операторами;
4. работа с вложенными объектами;
5. использование JavaScript-функций и курсоров;
6. выполнение агрегированных запросов;
7. редактирование документов;
8. удаление документов из коллекции;
9. создание ссылок между коллекциями;
10. создание, просмотр и удаление индексов;
11. анализ плана выполнения запроса с помощью explain.

Выполнение задания

1. Вставка документов в коллекцию unicorns

В начале выполнения работы была создана база данных learn. Затем была заполнена коллекция unicorns, содержащая документы с информацией о единорогах. Каждый документ содержит имя, список предпочтений, вес, пол и, при наличии, количество убитых вампиров.

Для добавления документов использовался метод insert. Документ единорога Dunx был предварительно записан в переменную document, после чего вставлен в коллекцию вторым способом.

```

learn> db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0ff91796bdc682cab115') }
}
learn> db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0ff92096bdc682cab116') }
}
learn> db.unicorns.insert({name: 'Unicrow', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0ff92896bdc682cab117') }
}
learn> db.unicorns.insert({name: 'Roooooodies', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0ff93596bdc682cab118') }
}
learn> db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0ff93e96bdc682cab119') }
}
learn> db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0ff95096bdc682cab11a') }
}
learn> db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0ff96296bdc682cab11b') }
}
learn> db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0ff96c96bdc682cab11c') }
}
learn> db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0ff97596bdc682cab11d') }
}
learn> db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0ff97d96bdc682cab11e') }
}
learn> db.unicorns.insert({name: 'Niaue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0ff98396bdc682cab11f') }
}
learn> document=({name: 'Duns', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165})
{
  name: 'Duns',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
learn> db.unicorns.insert(document)
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a0ff9ac96bdc682cab120') }
}

```

Рисунок 1 — Вставка документов в коллекцию *unicorns*.

```

learn> db.unicorns.find()
{
  {
    _id: ObjectId( '6a0ff91796bdc6682abc115' ),
    name: 'Turny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId( '6a0ff92096bdc6682abc116' ),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId( '6a0ff92096bdc6682abc117' ),
    name: 'Unicron',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId( '6a0ff92596bdc6682abc118' ),
    name: 'Boooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId( '6a0ff93e96bdc6682abc119' ),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId( '6a0ff95096bdc6682abc11a' ),
    name: 'Ayaa',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId( '6a0ff96296bdc6682abc11b' ),
    name: 'Tanny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId( '6a0ff96c96bdc6682abc11c' ),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId( '6a0ff97596bdc6682abc11d' ),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId( '6a0ff97d96bdc6682abc11e' ),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId( '6a0ff98396bdc6682abc11f' ),
    name: 'Ninue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId( '6a0ff9ac96bdc6682abc120' ),
    name: 'Toox',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
}
learn>

```

Рисунок 2 — Проверка содержимого коллекции *unicorns* с помощью метода *find*.

После выполнения команд в коллекции *unicorns* было создано 12 документов. Команда *find* позволила проверить содержимое коллекции.

2. Выборка данных из коллекции unicorns

В данном разделе были выполнены запросы на выборку документов из коллекции unicorns. Для получения данных использовались методы find и findOne. Для сортировки результата использовался метод sort, а для ограничения количества выводимых документов — метод limit.

2.1. Вывод самцов и самок единорогов

```
learn> db.unicorns.find((gender: '♂')).sort((name: 1))
[
  {
    _id: ObjectId('5a9f7909a6d040000000000000000000'),
    name: 'Arno',
    loves: [ 'grass', 'vegetables' ],
    weight: 704,
    gender: '♂',
    vampires: 165
  },
  {
    _id: ObjectId('5a9f7909a6d040000000000000000001'),
    name: 'Arno',
    loves: [ 'carrots', 'potatoes' ],
    weight: 600,
    gender: '♂',
    vampires: 63
  },
  {
    _id: ObjectId('5a9f7909a6d040000000000000000002'),
    name: 'Arno',
    loves: [ 'grass', 'leaves' ],
    weight: 690,
    gender: '♂',
    vampires: 39
  },
  {
    _id: ObjectId('5a9f7909a6d040000000000000000003'),
    name: 'Arno',
    loves: [ 'carrots', 'vegetables' ],
    weight: 650,
    gender: '♂',
    vampires: 54
  },
  {
    _id: ObjectId('5a9f7909a6d040000000000000000004'),
    name: 'Arno',
    loves: [ 'potatoes', 'leaves' ],
    weight: 421,
    gender: '♂',
    vampires: 2
  },
  {
    _id: ObjectId('5a9f7909a6d040000000000000000005'),
    name: 'Arno',
    loves: [ 'potatoes' ],
    weight: 579,
    gender: '♂',
    vampires: 99
  },
  {
    _id: ObjectId('5a9f7909a6d040000000000000000006'),
    name: 'Arno',
    loves: [ 'potatoes', 'vegetables' ],
    weight: 984,
    gender: '♂',
    vampires: 182
  }
]
learn> db.unicorns.find((gender: '♀')).sort((name: 1)).limit(3)
[
  {
    _id: ObjectId('5a9f7909a6d040000000000000000007'),
    name: 'Arno',
    loves: [ 'carrots', 'grass' ],
    weight: 450,
    gender: '♀',
    vampires: 43
  },
  {
    _id: ObjectId('5a9f7909a6d040000000000000000008'),
    name: 'Arno',
    loves: [ 'potatoes', 'leaves' ],
    weight: 733,
    gender: '♀',
    vampires: 40
  },
  {
    _id: ObjectId('5a9f7909a6d040000000000000000009'),
    name: 'Arno',
    loves: [ 'potatoes', 'vegetables' ],
    weight: 601,
    gender: '♀',
    vampires: 33
  }
]
learn> db.unicorns.findOne((gender: '♀', loves: 'carrots'))
{
  _id: ObjectId('5a9f7909a6d040000000000000000007'),
  name: 'Arno',
  loves: [ 'carrots', 'grass' ],
  weight: 450,
  gender: '♀',
  vampires: 43
}
learn> db.unicorns.find((gender: '♀', loves: 'carrots')).limit(1)
[
  {
    _id: ObjectId('5a9f7909a6d040000000000000000007'),
    name: 'Arno',
    loves: [ 'carrots', 'grass' ],
    weight: 450,
    gender: '♀',
    vampires: 43
  }
]
```

Рисунок 3 — Выборка самцов и самок единорогов с использованием sort, limit и findOne.

2.2. Исключение полей из результата

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('6a0ff91796bdcd682cabc115'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a0ff92896bdcd682cabc117'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a0ff93596bdcd682cabc118'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a0ff96296bdcd682cabc11b'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a0ff96c96bdcd682cabc11c'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a0ff97d96bdcd682cabc11e'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a0ff9ac96bdcd682cabc120'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  }
]
```

Рисунок 4 — Вывод списка самцов единорогов с исключением полей *loves* и *gender*.

2.3. Обратный порядок добавления и работа с массивом *loves*

```
learn> db.unicorns.find().sort({ $natural: -1 })
[
  {
    _id: ObjectId('6a0ff9ac96bdc682cab120'),
    name: 'Dunk',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0ff98396bdc682cab11f'),
    name: 'Ninus',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f',
    vampires: 1
  },
  {
    _id: ObjectId('6a0ff97d96bdc682cab11e'),
    name: 'Tilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a0ff97596bdc682cab11d'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a0ff96c96bdc682cab11c'),
    name: 'Raieigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a0ff96296bdc682cab11b'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0ff95096bdc682cab11a'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0ff93e96bdc682cab119'),
    name: 'Spinars',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a0ff93596bdc682cab118'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a0ff93896bdc682cab117'),
    name: 'Unicron',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a0ff92096bdc682cab116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0ff91796bdc682cab115'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  }
]
```

Рисунок 5 — Вывод документов в обратном порядке добавления.

```
learn> db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
```

Рисунок 6 — Вывод первого элемента массива loves.

В результате были выполнены разные виды выборки документов: с сортировкой, ограничением количества документов, поиском первого подходящего документа, исключением отдельных полей и выводом первого элемента массива loves.

3. Использование логических операторов

В данном разделе были выполнены запросы с использованием логических операторов MongoDB. Для ограничения диапазона значений использовались операторы \$gte и \$lte. Для проверки наличия нескольких элементов в массиве использовался оператор \$all. Для поиска документов без заданного поля использовался оператор \$exists.


```

learn> db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn>
learn>
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a0ff98396bdcd682cabc11f'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]

```

Рисунок 7 — Выполнение запросов с логическими операторами к коллекции *unicorns*.

С помощью логических операторов были найдены самки единорогов заданного веса, самцы с определёнными предпочтениями, документы без поля `vampires`, а также сформирован упорядоченный список самцов с первым элементом массива `loves`.

4. Работа с вложенными объектами

В данном разделе была создана коллекция `towns`, содержащая документы с вложенным объектом `mayor`. Вложенный объект хранит сведения о мэре города. Для обращения к полям вложенного объекта использовалась точечная запись.

```

learn> db.towns.insert({
  name: "Punxsutawney ",
  populatiuon: 6200,
  last_sensus: ISODate("2000-01-31"),
  famous_for: [""],
  mayor: {
    name: "Jim Wehrle"
  }
})
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a10029e96bdc6682cabc121') }
}
learn> db.towns.insert({
  name: "New York",
  populatiuon: 22200000,
  last_sensus: ISODate("2009-07-31"),
  famous_for: ["status of liberty", "food"],
  mayor: {
    name: "Michael Bloomberg",
    party: "I"
  }
})
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a1002a596bdc6682cabc122') }
}
learn> db.towns.insert({
  name: "Portland",
  populatiuon: 528000,
  last_sensus: ISODate("2009-07-28"),
  famous_for: ["beer", "food"],
  mayor: {
    name: "Sam Adams",
    party: "D"
  }
})
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a1002ab96bdc6682cabc123') }
}
learn> db.towns.find()
[
  {
    _id: ObjectId('6a10029e96bdc6682cabc121'),
    name: "Punxsutawney ",
    populatiuon: 6200,
    last_sensus: ISODate('2000-01-31T00:00:00.000Z'),
    famous_for: [ "" ],
    mayor: { name: "Jim Wehrle" }
  },
  {
    _id: ObjectId('6a1002a596bdc6682cabc122'),
    name: "New York",
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ "status of liberty", "food" ],
    mayor: { name: "Michael Bloomberg", party: "I" }
  },
  {
    _id: ObjectId('6a1002ab96bdc6682cabc123'),
    name: "Portland",
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-28T00:00:00.000Z'),
    famous_for: [ "beer", "food" ],
    mayor: { name: "Sam Adams", party: "D" }
  }
]
learn> db.towns.find(
  { "mayor.party": "I" },
  { name: 1, mayor: 1 }
)
[
  {
    _id: ObjectId('6a1002a596bdc6682cabc122'),
    name: "New York",
    mayor: { name: "Michael Bloomberg", party: "I" }
  }
]
learn> db.towns.find(
  { "mayor.party": { $exists: false } },
  { name: 1, mayor: 1 }
)
[
  {
    _id: ObjectId('6a10029e96bdc6682cabc121'),
    name: "Punxsutawney ",
    mayor: { name: "Jim Wehrle" }
  }
]

```

Рисунок 8 — Создание коллекции towns и выполнение запросов к вложенному объекту mayor.

С помощью условия "major.party": "I" был найден город с независимым мэром. С помощью оператора \$exists были найдены документы, где поле major.party отсутствует.

5. Использование JavaScript-функции и курсора

В данном разделе была создана JavaScript-функция для вывода списка самцов единорогов. Внутри функции был создан курсор, к которому применялись методы sort и limit. Для перебора документов курсора использовался метод forEach.

```
learn> function showMaleUnicorns() {  
|   var cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2);  
|   cursor.forEach(function(obj) {  
|     print(obj.name);  
|   });  
| }  
[Function: showMaleUnicorns]  
learn> showMaleUnicorns()  
Dunx  
Horny
```

Рисунок 9 — Использование JavaScript-функции, курсора, сортировки, limit и forEach.

В результате выполнения функции были выведены имена первых двух самцов единорогов в лексикографическом порядке.

6. Агрегированные запросы

В данном разделе были выполнены агрегированные запросы к коллекции unicorns. С помощью count было найдено количество самок единорогов заданного веса. С помощью distinct был получен список уникальных предпочтений. Для подсчёта количества особей каждого пола использовался метод aggregate.

```
learn> db.unicorns.find({gender: 'F', weight: {$gte: 500, $lte: 600}}).count()  
2  
learn> db.unicorns.distinct("loves")  
[  
  'apple',      'carrot',  
  'chocolate', 'energun',  
  'grape',      'lemon',  
  'papaya',     'redbull',  
  'strawberry', 'sugar',  
  'watermelon'  
]  
learn> db.unicorns.aggregate([  
|   {$group: {_id: "$gender", count: {$sum: 1}}}  
| ])  
[ { _id: 'm', count: 7 }, { _id: 'F', count: 5 } ]
```

Рисунок 10 — Выполнение агрегированных запросов к коллекции unicorns.

В результате было найдено количество самок весом от 500 до 600 кг, получен список уникальных значений массива loves и выполнена группировка документов по полу.

7. Редактирование данных

В данном разделе были выполнены операции редактирования документов в коллекциях unicorns и towns. Для изменения значений полей использовались операторы \$set, \$inc, \$unset, \$push, \$addToSet и \$each.

7.1. Добавление Barny, изменение Ayna и Raleigh

```
learn> db.unicorns.save({name: 'Barny', loves: [ 'grape' ], weight: 340, gender: 'M'})
TypeError: db.unicorns.save is not a function
learn> db.unicorns.insert({
  |   name: 'Barny',
  |   loves: ['grape'],
  |   weight: 340,
  |   gender: 'M'
  | })
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10052896bdcd682cabc124') }
}
learn> db.unicorns.find({name: 'Barny'})
[
  {
    _id: ObjectId('6a10052896bdcd682cabc124'),
    name: 'Barny',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'M'
  }
]
learn> db.unicorns.update(
  |   {name: 'Ayna'},
  |   {$set: {weight: 800, vampires: 51}}
  | )
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a0ff95096bdcd682cabc11a'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'F',
    vampires: 51
  }
]
learn> db.unicorns.update(
  |   {name: 'Raleigh'},
  |   {$set: {loves: [ 'redbull' ]}}
  | )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a0ff96c96bdcd682cabc11c'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'M',
    vampires: 2
  }
]
```

Рисунок 11 — Ошибка метода save, добавление Barny и изменение документов Ayna и Raleighunicorns.

7.2. Увеличение значения vampires всем самцам

```

learn> db.unicorns.update(
  {gender: 'w'},
  {$inc: {vampires: 5}},
  {multi: true}
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> db.unicorns.find(
  {gender: 'w'},
  {name: 1, gender: 1, vampires: 1}
)
[
  {
    _id: ObjectId('6a8ff91796bdcdb082cab115'),
    name: 'wenny',
    gender: 'w',
    vampires: 68
  },
  {
    _id: ObjectId('6a8ff92296bdcdb082cab117'),
    name: 'unicorn',
    gender: 'w',
    vampires: 187
  },
  {
    _id: ObjectId('6a8ff93596bdcdb082cab118'),
    name: 'Rosesandies',
    gender: 'w',
    vampires: 184
  },
  {
    _id: ObjectId('6a8ff96296bdcdb082cab11b'),
    name: 'kenny',
    gender: 'w',
    vampires: 44
  },
  {
    _id: ObjectId('6a8ff96c96bdcdb082cab11c'),
    name: 'malign',
    gender: 'w',
    vampires: 7
  },
  {
    _id: ObjectId('6a8ff97db6bdcdb082cab11e'),
    name: 'pilar',
    gender: 'w',
    vampires: 59
  },
  {
    _id: ObjectId('6a8ff9ac96bdcdb082cab120'),
    name: 'dunk',
    gender: 'w',
    vampires: 178
  },
  {
    _id: ObjectId('6a10052896bdcdb082cab124'),
    name: 'danny',
    gender: 'w',
    vampires: 5
  }
]

```

Рисунок 12 — Увеличение значения vampires всем самцам единорогов с использованием оператора \$inc.

Оператор \$inc был использован для увеличения числового значения vampires на 5. Параметр multi: true позволил применить изменение ко всем документам, соответствующим условию.

7.3. Изменение вложенного объекта и массивов

```

learn> db.towns.update(
|   {name: 'Portland'},
|   {$unset: {'mayor.party': 1}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name: 'Portland'})
[
  {
    _id: ObjectId('6a1002ab96bdc682cabc123'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn> db.unicorns.update(
|   {name: 'Pilot'},
|   {$push: {loves: 'chocolate'}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a0ff97d96bdc682cabc11e'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn> db.unicorns.update(
|   {name: 'Aurora'},
|   {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a0ff92096bdc682cabc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]

```

Рисунок 13 — Изменение вложенного объекта mayor и массивов loves

С помощью \$unset было удалено поле party из вложенного объекта mayor у города Portland. Оператор \$push добавил новое значение в массив loves у Pilot. Операторы \$addToSet и \$each были использованы для добавления нескольких значений в массив loves у Aurora без создания дублей.

8. Удаление данных из коллекции

В данном разделе была выполнена работа с удалением документов из коллекции towns. Сначала коллекция была заполнена документами городов, затем были удалены документы с беспартийными мэрами, а после этого коллекция была очищена полностью.

```
learn> db.towns.drop()
true
learn> db.towns.insert({
  name: "Punxsutawney ",
  popujatiuon: 6200,
  last_sensus: ISODate("2008-01-31"),
  famous_for: ["phil the groundhog"],
  mayor: {
    name: "Jim Wehrle"
  }
})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a100a9496bdc6682cabc128') }
}
learn> db.towns.insert({
  name: "New York",
  popujatiuon: 22200000,
  last_sensus: ISODate("2009-07-31"),
  famous_for: ["status of liberty", "food"],
  mayor: {
    name: "Michael Bloomberg",
    party: "I"
  }
})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a100a9e96bdc6682cabc129') }
}
learn> db.towns.insert({
  name: "Portland",
  popujatiuon: 528000,
  last_sensus: ISODate("2009-07-20"),
  famous_for: ["beer", "food"],
  mayor: {
    name: "Sam Adams",
    party: "D"
  }
})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a100a9e96bdc6682cabc12a') }
}
learn> db.towns.find()
[
  {
    _id: ObjectId('6a100a9496bdc6682cabc128'),
    name: "Punxsutawney ",
    popujatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ "phil the groundhog" ],
    mayor: { name: "Jim Wehrle" }
  },
  {
    _id: ObjectId('6a100a9e96bdc6682cabc129'),
    name: "New York",
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ "status of liberty", "food" ],
    mayor: { name: "Michael Bloomberg", party: "I" }
  },
  {
    _id: ObjectId('6a100a9e96bdc6682cabc12a'),
    name: "Portland",
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ "beer", "food" ],
    mayor: { name: "Sam Adams", party: "D" }
  }
]
learn> db.towns.remove({'mayor.party': {'$exists': false}})
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete, or bulkWrite.
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a100a9e96bdc6682cabc129'),
    name: "New York",
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ "status of liberty", "food" ],
    mayor: { name: "Michael Bloomberg", party: "I" }
  },
  {
    _id: ObjectId('6a100a9e96bdc6682cabc12a'),
    name: "Portland",
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ "beer", "food" ],
    mayor: { name: "Sam Adams", party: "D" }
  }
]
learn> db.towns.remove({})
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find()

learn> show collections
towns
unicorns
```

Рисунок 14 — Удаление документов из коллекции towns и очистка коллекции.

Для удаления документов с беспартийными мэрами использовался метод `remove` с условием `{"mayor.party": {$exists: false}}`. Затем команда `remove({})` была использована для очистки коллекции. В отличие от `drop`, команда `remove({})` удаляет документы, но не удаляет саму коллекцию.

9. Ссылки между коллекциями

В данном разделе была создана коллекция `habitats`, содержащая зоны обитания единорогов. В качестве идентификатора `_id` использовалось краткое название зоны. После этого нескольким документам коллекции `unicorns` было добавлено поле `habitat`, содержащее ссылку `DBRef` на документ из коллекции `habitats`.

```
learn> db.habitats.insert({_id: 'forest', name: 'Magical forest', description: 'Forest habitat for unicorns'})
{ acknowledged: true, insertedIds: { '_id': 'forest' } }
learn> db.habitats.insert({_id: 'mountain', name: 'Mystical mountain', description: 'Mountain habitat for unicorns'})
{ acknowledged: true, insertedIds: { '_id': 'mountain' } }
learn> db.habitats.insert({_id: 'lake', name: 'Silver lake', description: 'Lake habitat for unicorns'})
{ acknowledged: true, insertedIds: { '_id': 'lake' } }
learn> db.habitats.find()
[
  {
    _id: 'forest',
    name: 'Magical forest',
    description: 'Forest habitat for unicorns'
  },
  {
    _id: 'mountain',
    name: 'Mystical mountain',
    description: 'Mountain habitat for unicorns'
  },
  {
    _id: 'lake',
    name: 'Silver lake',
    description: 'Lake habitat for unicorns'
  }
]
learn> db.unicorns.update({name: 'Aurora'}, {$set: {habitat: {$ref: 'habitats', $id: 'forest'}}})
{ acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0 }
learn> db.unicorns.update({name: 'Unicorn'}, {$set: {habitat: {$ref: 'habitats', $id: 'mountain'}}})
{ acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0 }
learn> db.unicorns.update({name: 'Lala'}, {$set: {habitat: {$ref: 'habitats', $id: 'lake'}}})
{ acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0 }
learn> db.unicorns.update({name: 'Nana'}, {$set: {habitat: {$ref: 'habitats', $id: 'lake'}}})
{ acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0 }
learn> db.unicorns.find({habitat: {$exists: true}}, {name: 1, habitat: 1})
[
  {
    _id: ObjectId('6aef7928b4d042cabc110'),
    name: 'Aurora',
    habitat: DBRef('habitats', 'forest')
  },
  {
    _id: ObjectId('6aef7928b4d042cabc117'),
    name: 'Unicorn',
    habitat: DBRef('habitats', 'mountain')
  },
  {
    _id: ObjectId('6aef7928b4d042cabc11d'),
    name: 'Lala',
    habitat: DBRef('habitats', 'lake')
  },
  {
    _id: ObjectId('6aef7928b4d042cabc11e'),
    name: 'Nana',
    habitat: DBRef('habitats', 'lake')
  }
]
learn> aurora = db.unicorns.findOne({name: 'Aurora'})
{
  _id: ObjectId('6aef7928b4d042cabc110'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
  weight: 450,
  gender: 'f',
  vampires: 43,
  habitat: DBRef('habitats', 'forest')
}
learn> aurora.habitat
DBRef('habitats', 'forest')
learn> db.habitats.findOne(_id: aurora.habitat.oid)
{
  _id: 'forest',
  name: 'Magical forest',
  description: 'Forest habitat for unicorns'
}
learn> db[aurora.habitat.collection].findOne(_id: aurora.habitat.oid)
{
  _id: 'forest',
  name: 'Magical forest',
  description: 'Forest habitat for unicorns'
}
```

Рисунок 15 — Создание коллекции `habitats` и добавление ссылок на зоны обитания в документы единорогов.

Проверка показала, что в документах коллекции unicorns появилось поле habitat. По данным ссылки был получен связанный документ из коллекции habitats.

10. Работа с индексами

В данном разделе была выполнена настройка индексов в коллекции unicorns. Для поля name был создан уникальный индекс. Затем была выполнена проверка работы ограничения уникальности и операции управления индексами.

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.ensureIndex({name: 1}, {"unique": true})
[ 'name_1' ]
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.insert({name: 'Aurora', loves: ['apple'], weight: 400, gender: 'f'})
Uncaught:
MongoBulkWriteError: E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name: "Aurora" }
Result: BulkWriteResult {
  insertedCount: 0,
  matchedCount: 0,
  modifiedCount: 0,
  deletedCount: 0,
  upsertedCount: 0,
  upsertedIds: {},
  insertedIds: {}
}
Write Errors: [
  WriteError {
    err: {
      index: 0,
      code: 11000,
      errmsg: 'E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name: "Aurora" }',
      errInfo: undefined,
      op: {
        name: 'Aurora',
        loves: [ 'apple' ],
        weight: 400,
        gender: 'f',
        _id: ObjectId('6a10425496bdc682cabc12d')
      }
    }
  }
]
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a0ff92096bdc682cabc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43,
    habitat: DBRef('habitats', 'forest')
  }
]
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

Рисунок 16 — Создание уникального индекса по полю name и управление индексами коллекции unicorns.

В результате было найдено количество самок весом от 500 до 600 кг, получен список уникальных значений массива `loves` и выполнена группировка документов по полу.

11. План запроса

В данном разделе была создана коллекция `numbers`, содержащая 100000 документов. Каждый документ содержит поле `value` со значениями от 0 до 99999. После этого был выполнен запрос на выбор последних четырёх документов и проведён анализ плана запроса до и после создания индекса по полю `value`.

11.1. Создание коллекции `numbers`

```
learn> db.numbers.drop()
true
learn> for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a10435196bdc682cad47cd') }
}
learn> db.numbers.count()
DeprecationWarning: Collection.count() is deprecated. Use countDocuments or estimatedDocumentCount.
100000
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a10435196bdc682cad47cd'), value: 99999 },
  { _id: ObjectId('6a10435196bdc682cad47cc'), value: 99998 },
  { _id: ObjectId('6a10435196bdc682cad47cb'), value: 99997 },
  { _id: ObjectId('6a10435196bdc682cad47cn'), value: 99996 }
]
```

Рисунок 17 — Создание коллекции `numbers` и выбор последних четырёх документов.

В результате была создана коллекция `numbers` из 100000 документов. Запрос с сортировкой по `value` в порядке убывания вернул последние четыре значения: 99999, 99998, 99997 и 99996.

11.2. План запроса до создания индекса

```
learn> db.numbers.explain('executionStats').find().sort({value: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexedFilterSet: false,
    indexFilterSet: false,
    queryHash: '46c7baf2',
    planCacheShapeHash: '46c7baf2',
    planCacheKey: '46c7baf2',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'COLLSCAN',
      inputStage: { stage: 'COLLSCAN', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 70,
  }
}
```

Рисунок 18 — План выполнения запроса до создания индекса по полю `value`.

До создания индекса значение `executionTimeMillis` составило 70 мс. В плане выполнения использовались стадии `COLLSCAN` и `SORT`, то есть MongoDB просматривала все документы коллекции и затем выполняла сортировку.

11.3. Создание индекса по полю `value` и запрос после создания индекса

```
learn> db.numbers.ensureIndex({value: 1})
[ 'value_1' ]
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a10435196bdc682cad47cd'), value: 99999 },
  { _id: ObjectId('6a10435196bdc682cad47cc'), value: 99998 },
  { _id: ObjectId('6a10435196bdc682cad47cb'), value: 99997 },
  { _id: ObjectId('6a10435196bdc682cad47ca'), value: 99996 }
]
learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
  }
}
```

Рисунок 19 — Создание индекса по полю `value` в коллекции `numbers` и план выполнения запроса.

После создания индекса в списке индексов коллекции `numbers` появился индекс `value_1`, а значение `executionTimeMillis` стало равно 0 мс. В

плане выполнения появилась стадия IXSCAN, указывающая на использование индекса. Количество просмотренных документов уменьшилось со 100000 до 4. Следовательно, индекс по полю value позволил выполнить запрос значительно эффективнее.

Выводы

В ходе выполнения лабораторной работы были изучены основные приёмы работы с документной базой данных MongoDB. Была создана база данных learn и несколько коллекций, предназначенных для демонстрации разных возможностей MongoDB.

В коллекции unicorns были выполнены операции вставки, выборки, изменения и удаления данных. Были использованы простые условия выборки, сортировка, ограничение количества результатов, проекции полей, работа с массивами и логические операторы. Также были выполнены запросы к вложенным объектам в коллекции towns и создана JavaScript-функция для работы с курсором.

В ходе работы были выполнены агрегированные запросы с использованием count, distinct и aggregate. Также были изучены способы изменения документов с помощью операторов \$set, \$inc, \$unset, \$push, \$addToSet и \$each.

Отдельно была рассмотрена работа со ссылками между коллекциями на примере коллекций unicorns и habitats. Для нескольких единорогов были добавлены ссылки DBRef на документы зон обитания.

В заключительной части работы были созданы и проверены индексы. Уникальный индекс по полю name позволил запретить добавление документов с повторяющимся именем. На коллекции numbers был проведён анализ плана запроса до и после создания индекса по полю value. После создания индекса время выполнения запроса уменьшилось с 70 мс до 0 мс, а план выполнения стал использовать индексную стадию IXSCAN вместо полного просмотра коллекции.